day4_v2

August 5, 2026

1 Day 4 — Lag x liquidity x participation: execution timing audit (v2, memory-safe)

Scope: month-end formation, execution starting only after a lag of L business days, names worked in liquidity order under a daily participation cap. Output is **timing only** (when each name gets done) – no execution cost, no intraday, no clustering, no NAV. AUM 1000 , lag grid 0-20bd, cap grid 2-30%.

This is a rewrite of the previous pass, which exhausted memory before producing any output. The engine and the analysis are unchanged; what changed is the data handling, brought in line with Day 3's conventions: the returns file is read once instead of twice and only for the columns actually used, everything is filtered to the `held` code set before any wide reshape, the synthetic-data fallback is gone, dense panels are `float32` where accumulation isn't a precision risk, every wide pandas intermediate is explicitly `del`'d once it has been converted to a numpy array, figures are closed after they're saved, and a running peak-memory printout follows each heavy section so a future run pinpoints exactly where memory goes if it ever happens again.

[1]:
```
from pathlib import Path
import gc
import resource
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

pd.set_option("display.width", 220); pd.set_option("display.max_columns", 40);␣
↪pd.set_option("display.max_rows", 200)

DATA_DIR = Path("/home/ubuntu/work/data")
OUT_DIR  = Path("/home/ubuntu/work/output"); OUT_DIR.mkdir(parents=True,␣
↪exist_ok=True)
UNIVERSE_FILE  = DATA_DIR / "jp_top500_universe_201001_202607.parquet"
RETURNS_FILE   = DATA_DIR / "daily_returns_20100104_20260729.parquet"
PORTFOLIO_FILE = DATA_DIR / "jp_top500_monthly_bpr_portfolios_201001_202606.csv"

COLS = dict(ret_date="base_ymd", ret_code="stock_code",␣
↪ret_value="daily_return",
            ret_price="price_close", ret_volume="volume",
            uni_date="date_eom", uni_code="code", uni_cap="mkt_cap",
```

```
            prt_date="DATE_EOM", prt_code="CODE", prt_weight="WEIGHT")
OI = ["#0072B2", "#009E73", "#E69F00", "#D55E00", "#56B4E9", "#CC79A7",␣
 ↪"#F0E442", "#000000"]
DTYPE = np.float32   # daily panels only, no accumulation risk -- halves their␣
 ↪footprint vs float64

AUM            = 100_000_000_000          # 1000
LAG_GRID       = [0, 2, 5, 7, 10, 13, 15, 17, 20]             # business days␣
 ↪after the month-end decision
CAP_GRID       = [0.02, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30]   # own share of␣
 ↪total daily volume
BASE_LAG, BASE_CAP = 2, 0.10
ADV_WIN        = 20                       # trailing window for ADV and for the␣
 ↪liquidity quintiles
MAX_EXEC_DAYS  = 500                      # give up on a name after this many␣
 ↪bd from its own start
WAVE_WAIT_BD   = 20                       # a liquidity stage waits at most␣
 ↪this long for the previous stage
BUCKET_NAMES   = ["Q1 illiquid", "Q2", "Q3", "Q4", "Q5 liquid"]   # index 0..4,␣
 ↪4 = most liquid

def peak_mb() -> float:
    """Peak resident set size for this process so far, in MB. Cheap,␣
 ↪stdlib-only -- print this after
    every heavy cell so a future OOM shows exactly where memory went, instead␣
 ↪of somewhere in the notebook."""
    return resource.getrusage(resource.RUSAGE_SELF).ru_maxrss / 1024

print(f"AUM={AUM:,}  lags={LAG_GRID}  caps={[f'{c:.0%}' for c in CAP_GRID]} ␣
 ↪panel dtype={DTYPE.__name__}")
print(f"peak RSS at notebook start: {peak_mb():,.0f} MB")
```

```
AUM=100,000,000,000  lags=[0, 2, 5, 7, 10, 13, 15, 17, 20]  caps=['2%', '5%',
'10%', '15%', '20%', '25%', '30%']  panel dtype=float32
peak RSS at notebook start: 199 MB
```

1.1 1. Load (Day 3 conventions, single pass)

Same `read`/`tidy` helpers as Day 3, with two changes that follow directly from Day 3's own "only load what you'll use" logic:

- **Read parquet columns selectively.** `read()` now takes an optional `columns=`, and `tidy()` always passes its own three key columns, so a file with extra columns never gets fully materialised.
- **Read the returns file once, not twice.** Day 3 calls `read(RETURNS_FILE)` twice – once via `tidy()` for `daily_return`, once more for `yen_volume`. This notebook's timing engine never uses `daily_return`, so the file is read once, for `yen_volume` only, and filtered down to

the **held** code set (`PRT  UNI`, the same definition Day 3 uses) before anything wider than a single column touches it. Held codes are worked out first, from the two small files, specifically so the ~20M-row returns file is never read without that filter already in hand.

No synthetic-data fallback here – Day 3 doesn't have one either; if a file is missing this fails loudly on `read()` / the `tidy()` assert rather than silently substituting random data.

file	columns used	note
`daily_returns_*.parquet`	`base_ymd`, `stock_code`, `price_close`, `volume`	only `price_close * volume` is used; `daily_return` is never loaded
`jp_top500_universe_*.parquet`	`date_eom`, `code`, `mkt_cap`	monthly top-500 membership, used for the liquidity quintiles
`jp_top500_monthly_bpr_portfolio_*.csv`	`DATE_EOM`, `CODE`, `WEIGHT`	~198 month-ends x ~100 names

[2]:
```
def read(path: Path, columns=None) -> pd.DataFrame:
    if path.suffix == ".parquet":
        return pd.read_parquet(path, columns=columns)
    return pd.read_csv(path, usecols=columns)

def tidy(path: Path, keys: tuple, name: str, dedupe: bool = False) -> pd.
 ↪DataFrame:
    date_col, code_col, val_col = keys
    df = read(path, columns=list(keys))
    missing = [c for c in keys if c not in df.columns]
    assert not missing, f"{path.name}: missing {missing}. Present: {list(df.
 ↪columns)}"
    out = pd.DataFrame({
        "date": pd.to_datetime(df[date_col].astype(str)) if str(df[date_col].
 ↪dtype).startswith(("int", "Int")) else pd.to_datetime(df[date_col]),
        "code": df[code_col].astype(str).str.strip().str.replace(r"\.0$", "",␣
 ↪regex=True).str.upper(),
        name: pd.to_numeric(df[val_col], errors="coerce"),
    }).dropna()
    return out.groupby(["date", "code"], as_index=False)[name].sum() if dedupe␣
 ↪else out

UNI = tidy(UNIVERSE_FILE, (COLS["uni_date"], COLS["uni_code"],␣
 ↪COLS["uni_cap"]), "cap")
PRT = tidy(PORTFOLIO_FILE, (COLS["prt_date"], COLS["prt_code"],␣
 ↪COLS["prt_weight"]), "weight", dedupe=True)

# held codes, decided before the returns file is touched -- same union Day 3␣
 ↪uses (PRT  UNI)
```

```
held = set(PRT["code"]) | set(UNI["code"])
print(f"held codes (PRT union UNI): {len(held)}")

# yen volume: read RETURNS_FILE exactly once, 4 columns only, filtered to␣
 ↪`held` before any reshape
_raw   = read(RETURNS_FILE, columns=[COLS["ret_date"], COLS["ret_code"],␣
 ↪COLS["ret_price"], COLS["ret_volume"]])
_codes = _raw[COLS["ret_code"]].astype(str).str.strip().str.replace(r"\.0$",␣
 ↪"", regex=True).str.upper()
_keep  = _codes.isin(held).to_numpy()
_dcol  = _raw[COLS["ret_date"]]
VOL = pd.DataFrame({
    "date": pd.to_datetime(_dcol.astype(str)[_keep]) if str(_dcol.dtype).
 ↪startswith(("int", "Int")) else pd.to_datetime(_dcol[_keep]),
    "code": _codes[_keep],
    "yen_volume": pd.to_numeric(_raw.loc[_keep, COLS["ret_price"]],␣
 ↪errors="coerce")
                * pd.to_numeric(_raw.loc[_keep, COLS["ret_volume"]],␣
 ↪errors="coerce"),
}).dropna()
VOL = VOL.groupby(["date", "code"], as_index=False)["yen_volume"].sum()
n_rows_before = len(_raw)
del _raw, _codes, _keep, _dcol
gc.collect()
print(f"returns file: {n_rows_before:,} rows read once, dropped to {len(VOL):,}␣
 ↪after the held-code filter")

CAL = pd.DatetimeIndex(np.sort(VOL["date"].unique()))
assert len(CAL) > 100 and CAL.is_monotonic_increasing

print(f"calendar : {len(CAL)} trading days, {CAL.min().date()} -> {CAL.max().
 ↪date()}")
print(f"volume   : {VOL['code'].nunique()} names (post held-filter), median␣
 ↪daily yen volume {VOL['yen_volume'].median():,.0f}")
print(f"universe : {UNI['date'].nunique()} month-ends x {UNI.groupby('date').
 ↪size().median():.0f} names")
print(f"portfolio: {PRT['date'].nunique()} month-ends x {PRT.groupby('date').
 ↪size().median():.0f} names, "
      f"weight sum per date min/max {PRT.groupby('date')['weight'].sum().min():.
 ↪4f}/{PRT.groupby('date')['weight'].sum().max():.4f}")
assert (PRT["weight"] >= 0).all(), "negative portfolio weights: long-only␣
 ↪assumption broken"
assert (VOL["yen_volume"] >= 0).all()
print(f"peak RSS so far: {peak_mb():,.0f} MB")
```

```
held codes (PRT union UNI): 939
```

```
returns file: 12,304,546 rows read once, dropped to 3,187,828 after the held-
code filter
calendar : 4054 trading days, 2010-01-04 -> 2026-07-29
volume   : 939 names (post held-filter), median daily yen volume 931,944,000
universe : 198 month-ends x 500 names
portfolio: 198 month-ends x 100 names, weight sum per date min/max 1.0000/1.0000
peak RSS so far: 2,456 MB
```

1.2 2. Panels: volume, trailing ADV, liquidity quintiles

Dense day x name panels, `held` codes only. Two dtype choices, made for different reasons:

- `VOLMAT` and `ADVMAT` are `float32` – daily values, no accumulation, and yen volumes carry nowhere near float32's ~7-digit precision limit, so this halves their footprint for free.
- `CUMV0` (cumulative volume with a leading zero row) stays `float64`. It's a running sum over ~4,000 days, and float32 error compounds with every addition – at the tens-of-trillions-of-yen scale a full history can reach, that is enough absolute error to shift a completion-day lookup by a day or more. `CUMV0` is small either way (tens of MB here), so there is no memory reason to touch it.

`ADVMAT[t, i]` is the trailing `ADV_WIN`-day median yen volume, shifted one day so a decision date only sees data strictly before it. The pandas `.where()/.rolling()/.shift()` chain is dropped (`del` + `gc.collect()`) the moment it has been converted to the numpy array, since pandas keeps every link in that chain alive for as long as anything downstream still references the frame.

Liquidity quintiles are cut cross-sectionally at each month-end on that month's `UNI` membership's trailing ADV; a name with no ADV history gets bucket 0 (most conservative). Bucket 4 = most liquid. Buckets are re-formed monthly.

[3]:
```
CODES = np.array(sorted(set(VOL["code"])))      # already `held`, and already␣
 ↪has volume data
CPOS  = pd.Series(np.arange(len(CODES)), index=CODES); NC = len(CODES)
DPOS  = pd.Series(np.arange(len(CAL)), index=CAL)

VOLW   = VOL.pivot_table(index="date", columns="code", values="yen_volume",␣
 ↪aggfunc="first").reindex(CAL).reindex(columns=CODES)
VOLMAT = np.nan_to_num(VOLW.to_numpy(dtype=DTYPE), nan=0.0)
CUMV0  = np.asfortranarray(np.vstack([np.zeros((1, NC)), np.cumsum(VOLMAT.
 ↪astype(np.float64), axis=0)]))   # (T+1, NC), float64 -- see markdown above

_adv_src = VOLW.where(VOLW > 0)
ADVMAT = _adv_src.rolling(ADV_WIN, min_periods=5).median().shift(1).
 ↪to_numpy(dtype=DTYPE)
del VOLW, _adv_src
gc.collect()

def snap(dates) -> pd.DatetimeIndex:
    pos = CAL.get_indexer(pd.DatetimeIndex(pd.to_datetime(dates)),␣
 ↪method="ffill")
```

```
    assert (pos >= 0).all(), "a date falls before the start of the trading␣
 ↪calendar"
    return CAL[pos]

def weight_vectors(w: pd.DataFrame) -> dict:
    out = {}
    for d, g in w.groupby("date"):
        v = np.zeros(NC)
        j = CPOS.reindex(g["code"]).to_numpy(dtype=float); ok = ~np.isnan(j)
        v[j[ok].astype(int)] = g["weight"].to_numpy()[ok]
        if v.sum() > 0:
            out[snap([d])[0]] = v / v.sum()
    return out

WVEC  = weight_vectors(PRT)
FORMS = pd.DatetimeIndex(sorted(WVEC.keys()))

UNI_D = pd.DatetimeIndex(np.sort(UNI["date"].unique()))
UNI_MEM = {d: CPOS.reindex(g["code"]).dropna().to_numpy(dtype=int) for d, g in␣
 ↪UNI.groupby("date")}
def universe_at(f):
    k = UNI_D.searchsorted(f, side="right") - 1
    return UNI_MEM[UNI_D[k]] if k >= 0 else np.arange(NC)

BUCKET = {}
for f in FORMS:
    adv = ADVMAT[int(DPOS[f])]
    mem = universe_at(f); mem = mem[np.isfinite(adv[mem]) & (adv[mem] > 0)]
    b = np.zeros(NC, dtype=int)
    if mem.size >= 25:
        cuts = np.quantile(adv[mem], [0.2, 0.4, 0.6, 0.8])
        ok = np.isfinite(adv) & (adv > 0)
        b[ok] = np.searchsorted(cuts, adv[ok], side="right")
    BUCKET[f] = np.clip(b, 0, 4)

f = FORMS[len(FORMS) // 2]
adv, bk = ADVMAT[int(DPOS[f])], BUCKET[f]
print(f"liquidity quintiles at {f.date()} (trailing {ADV_WIN}d median yen␣
 ↪volume):")
for q in range(5):
    sel = (bk == q) & np.isfinite(adv)
    print(f"  {BUCKET_NAMES[q]:<12} n={sel.sum():4d}  median ADV {np.
 ↪nanmedian(adv[sel]):>18,.0f}")

panel_mb = (VOLMAT.nbytes + CUMV0.nbytes + ADVMAT.nbytes) / 1e6
print(f"\npanels: {VOLMAT.shape[0]} days x {NC} names, {panel_mb:.1f} MB total␣
 ↪(VOLMAT+CUMV0+ADVMAT); {len(FORMS)} formation dates")
```

```
print(f"peak RSS so far: {peak_mb():,.0f} MB")
```

```
liquidity quintiles at 2018-04-27 (trailing 20d median yen volume):
  Q1 illiquid  n= 363  median ADV        357,188,864
  Q2           n= 140  median ADV      1,063,104,000
  Q3           n= 109  median ADV      1,900,409,344
  Q4           n= 101  median ADV      3,326,800,384
  Q5 liquid    n= 101  median ADV      8,241,924,608

panels: 4054 days x 939 names, 60.9 MB total (VOLMAT+CUMV0+ADVMAT); 198
formation dates
peak RSS so far: 2,456 MB
```

1.3 3. Timing engine

One rule, applied per name: on any day the fund may take at most `cap` of that day's **total** volume including its own print, i.e. at most `cap/(1-cap)` of the printed market volume. So a target trade of `X` yen needs cumulative market volume of `X*(1-cap)/cap` from the day it starts. Completion day is therefore read straight off the cumulative volume column with a binary search, no day loop, which also makes it structurally impossible for a name to trade before its start index.

Ordering (`order`):

- `liquid_first` (the task convention): Q5 starts at decision + L; each next quintile starts the business day after the previous quintile has fully completed, or after `WAVE_WAIT_BD` days, whichever comes first, so one stuck illiquid tail cannot stall the schedule indefinitely.
- `parallel`: every name starts at decision + L (control).
- `illiquid_first`: reverse waves (control; this is the order that minimises total completion time, so it is the natural benchmark for what liquid-first costs in calendar time).

Trades are the raw month-over-month weight change `w_new - w_old` times AUM, undrifted, consistent with Day 3.

[4]:
```
def shift_bd_idx(f, n: int):
    i = int(DPOS[snap([f])[0]]) + n
    return i if 0 <= i < len(CAL) else None

def completion_idx(need_vol, cols, start_idx, max_exec=MAX_EXEC_DAYS):
    """Smallest calendar index t >= start_idx at which cumulative market volume␣
 ↪since start_idx
    covers need_vol.
      t >= 0 : completed on that day
      -1     : the cap is genuinely binding, not done within max_exec days of a␣
 ↪full window
      -2     : the allowed window runs past the end of the sample, so we cannot␣
 ↪tell (truncated)"""
    out = np.full(len(cols), -1, dtype=int)
    truncated = start_idx + max_exec > len(CAL) - 1
    lim = min(start_idx + max_exec, len(CAL))
```

```
    for k in range(len(cols)):
        col = CUMVO[:, cols[k]]
        idx = int(np.searchsorted(col, col[start_idx] + need_vol[k],␣
 ↪side="left"))
        t = idx - 1
        if idx <= len(CAL) - 1 and start_idx <= t < lim:
            out[k] = t
        elif truncated:
            out[k] = -2
    return out

def month_paths(delta_w, bucket_of, i_dec, lag, cap, order="liquid_first",
                max_exec=MAX_EXEC_DAYS, wave_wait=WAVE_WAIT_BD):
    act = np.flatnonzero(delta_w != 0)
    s0 = i_dec + lag
    if act.size == 0 or s0 >= len(CAL):
        return None
    need = np.abs(delta_w[act]) * AUM * (1.0 - cap) / cap
    bk = bucket_of[act]
    start = np.full(act.size, s0, dtype=int); finish = np.full(act.size, -1,␣
 ↪dtype=int)
    if order == "parallel":
        groups = [np.arange(act.size)]
    else:
        seq = [4, 3, 2, 1, 0] if order == "liquid_first" else [0, 1, 2, 3, 4]
        groups = [np.flatnonzero(bk == b) for b in seq]
    s = s0
    for g in groups:
        if g.size == 0:
            continue
        start[g] = s
        fin = completion_idx(need[g], act[g], s, max_exec)
        finish[g] = fin
        done = fin >= 0
        stage_end = int(fin[done].max()) if done.any() else min(s + max_exec,␣
 ↪len(CAL) - 1)
        s = int(min(stage_end + 1, s + wave_wait, len(CAL) - 1))
    return act, start, finish, bk, need

def run_combo(lag, cap, order="liquid_first", detail=False):
    """Runs every month. Returns per-bucket timing stats; detail=True also␣
 ↪returns the name-month frame."""
    parts = []
    for m in range(1, len(FORMS)):
        d1 = FORMS[m]; i_dec = int(DPOS[d1])
        res = month_paths(WVEC[d1] - WVEC[FORMS[m - 1]], BUCKET[d1], i_dec,␣
 ↪lag, cap, order)
```

```
        if res is None:
            continue
        act, start, finish, bk, need = res
        parts.append(pd.DataFrame({
            "decision": d1, "code": CODES[act], "bucket": bk,
            "start_offset_bd": start - i_dec,
            "days_exec": np.where(finish >= 0, finish - start + 1, np.nan),
            "days_from_decision": np.where(finish >= 0, finish - i_dec + 1, np.
↪nan),
            "trade_yen": need * cap / (1.0 - cap),
            "truncated": finish == -2,
        }))
    D_all = pd.concat(parts, ignore_index=True)
    del parts
    trunc = D_all.groupby("bucket")["truncated"].mean().reindex(range(5))
    D = D_all[~D_all["truncated"]].copy()          # sample-end cases carry no␣
↪information about capacity
    g = D.groupby("bucket")
    S = pd.DataFrame({
        "n": g.size(),
        "mean_start_offset_bd": g["start_offset_bd"].mean(),
        "mean_days_exec": g["days_exec"].mean(),
        "median_days_exec": g["days_exec"].median(),
        "p90_days_exec": g["days_exec"].quantile(0.90),
        "mean_days_from_decision": g["days_from_decision"].mean(),
        "median_days_from_decision": g["days_from_decision"].median(),
        "share_unresolved": g["days_exec"].apply(lambda s: s.isna().mean()),
        "share_truncated_by_sample_end": trunc,
        "median_trade_yen": g["trade_yen"].median(),
    }).reindex(range(5))
    S.index = pd.Index([BUCKET_NAMES[b] for b in S.index], name="bucket")
    S.insert(0, "cap", cap); S.insert(0, "lag_bd", lag); S.insert(0, "order",␣
↪order)
    if detail:
        return S, D_all
    del D_all, D, g
    return S
```

1.4 4. Lag logic audit

The task convention: the portfolio is decided on month-end information, and **nothing may trade before that month-end**. Execution then starts L business days later, working the most liquid quintile first.

Checks:

- **A1** every formation date is the last trading day of its month.
- **A2** `shift_bd_idx(f, L)` lands exactly L trading days after `f` in the calendar index.

- **A3** no name's start index is earlier than `index(f) + L`, for every month, every lag, every order.
- **A4** under liquid-first ordering the mean start offset is monotonically non-decreasing from Q5 to Q1.
- **A5** completion duration is weakly decreasing in the participation cap, within each bucket.
- **A6** a worked example: one month printed line by line, so the dates can be eyeballed.

[5]:
```
# A1 formation dates are the last trading day of their month
eom_of_month = pd.Series(CAL, index=CAL).groupby(CAL.to_period("M")).max()
bad_eom = [f for f in FORMS if eom_of_month.get(f.to_period("M")) != f]
print(f"A1 formation dates that are NOT the month's last trading day:␣
 ↪{len(bad_eom)}" + ("" if not bad_eom else f"  {bad_eom[:5]}"))

# A2 lag arithmetic is exact in trading days
a2 = all(int(DPOS[CAL[shift_bd_idx(f, L)]]) - int(DPOS[snap([f])[0]]) == L
         for f in FORMS[:-1] for L in LAG_GRID if shift_bd_idx(f, L) is not␣
 ↪None)
print(f"A2 shift_bd_idx lands exactly L trading days after the decision: {a2}")

# A3 nothing starts before decision + L, for every month / lag / order
viol = 0
for order in ["liquid_first", "parallel", "illiquid_first"]:
    for L in LAG_GRID:
        for m in range(1, len(FORMS)):
            d1 = FORMS[m]; i_dec = int(DPOS[d1])
            r = month_paths(WVEC[d1] - WVEC[FORMS[m - 1]], BUCKET[d1], i_dec,␣
 ↪L, 0.30, order)
            if r is None:
                continue
            viol += int((r[1] < i_dec + L).sum())
print(f"A3 name-months starting before decision + lag: {viol}  (must be 0)")
assert viol == 0

# A4 liquid-first really does start the liquid quintile first
S_lf = run_combo(BASE_LAG, BASE_CAP, "liquid_first")
so = S_lf["mean_start_offset_bd"].reindex(BUCKET_NAMES[::-1])
print(f"A4 mean start offset (bd from decision), Q5 -> Q1: {so.round(2).
 ↪to_dict()}")
print(f"   non-decreasing from liquid to illiquid: {bool(np.all(np.diff(so.
 ↪dropna().to_numpy()) >= -1e-9))}")

# A5 more capacity -> weakly faster, within each bucket
mono = run_combo(BASE_LAG, 0.02)["mean_days_exec"], run_combo(BASE_LAG, 0.
 ↪30)["mean_days_exec"]
print(f"A5 mean exec days at cap 2% vs 30%:")
print(pd.DataFrame({"cap 2%": mono[0], "cap 30%": mono[1]}).round(1).
 ↪to_string())
```

```
# A6 one month, printed
m = len(FORMS) // 2; d0, d1 = FORMS[m - 1], FORMS[m]; i_dec = int(DPOS[d1])
act, start, finish, bk, need = month_paths(WVEC[d1] - WVEC[FORMS[m - 1]],␣
 ↪BUCKET[d1], i_dec, BASE_LAG, BASE_CAP)
ex = pd.DataFrame({"code": CODES[act], "bucket": [BUCKET_NAMES[b] for b in bk],
                   "trade_yen": need * BASE_CAP / (1 - BASE_CAP),
                   "start_date": CAL[start], "finish_date": [CAL[t] if t >= 0␣
 ↪else pd.NaT for t in finish],
                   "start_offset_bd": start - i_dec,
                   "days_exec": [t - s + 1 if t >= 0 else np.nan for t, s in␣
 ↪zip(finish, start)]})
print(f"\nA6 worked example  decision (month-end) {d1.date()}  ->  earliest␣
 ↪allowed trade date {CAL[i_dec + BASE_LAG].date()}  (lag {BASE_LAG}bd, cap␣
 ↪{BASE_CAP:.0%})")
print(ex.sort_values(["bucket", "trade_yen"], ascending=[False, False]).
 ↪groupby("bucket", observed=True).head(2).to_string(index=False))
print("\nstage start dates by quintile:")
print(ex.groupby("bucket", observed=True)["start_date"].min().
 ↪sort_index(ascending=False).to_string())
print(f"\npeak RSS so far: {peak_mb():,.0f} MB")
```

```
A1 formation dates that are NOT the month's last trading day: 0
A2 shift_bd_idx lands exactly L trading days after the decision: True
A3 name-months starting before decision + lag: 0  (must be 0)
A4 mean start offset (bd from decision), Q5 -> Q1: {'Q5 liquid': 2.0, 'Q4':
5.13, 'Q3': 9.47, 'Q2': 14.26, 'Q1 illiquid': 22.23}
   non-decreasing from liquid to illiquid: True
A5 mean exec days at cap 2% vs 30%:
             cap 2%  cap 30%
bucket
Q1 illiquid    12.2      1.4
Q2              5.5      1.1
Q3              4.1      1.1
Q4              4.0      1.1
Q5 liquid       2.9      1.0

A6 worked example  decision (month-end) 2018-04-27  ->  earliest allowed trade
date 2018-05-02  (lag 2bd, cap 10%)
code      bucket    trade_yen start_date finish_date  start_offset_bd  days_exec
6971   Q5 liquid 1.827725e+09 2018-05-02  2018-05-07                2          2
8766   Q5 liquid 1.658760e+08 2018-05-02  2018-05-02                2          1
8795          Q4 8.915776e+08 2018-05-08  2018-05-11                4          4
9831          Q4 4.332231e+08 2018-05-08  2018-05-09                4          2
9532          Q3 6.073235e+07 2018-05-14  2018-05-14                8          1
7182          Q3 4.866054e+07 2018-05-14  2018-05-14                8          1
3941          Q2 1.993942e+08 2018-05-15  2018-05-16                9          2
```

```
9107          Q2 1.855946e+08 2018-05-15  2018-05-16                9          2
7282 Q1 illiquid 2.592945e+08 2018-05-17  2018-05-21               11          3
9075 Q1 illiquid 1.992494e+08 2018-05-17  2018-05-22               11          4

stage start dates by quintile:
bucket
Q5 liquid     2018-05-02
Q4            2018-05-08
Q3            2018-05-14
Q2            2018-05-15
Q1 illiquid   2018-05-17

peak RSS so far: 2,456 MB
```

1.5 5. Base case: how long each liquidity bucket takes

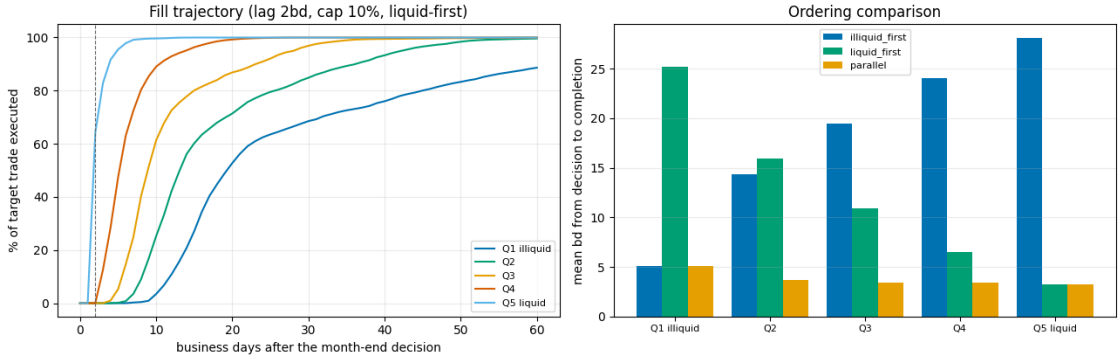
Base combination is lag 2bd, cap 10%, liquid-first. Three views: the per-bucket table, the aggregate fill trajectory (share of the month's target trade completed by day h after the decision), and the ordering comparison.

[6]:
```
S_base, D_base = run_combo(BASE_LAG, BASE_CAP, "liquid_first", detail=True)
print(f"base case: AUM {AUM:,.0f}, lag {BASE_LAG}bd, cap {BASE_CAP:.0%},␣
 ↪liquid-first")
print(S_base.drop(columns=["order", "lag_bd", "cap"]).round(2).to_string())

# fill trajectory: share of target trade value completed by day h after the␣
 ↪decision
H = np.arange(0, 61)
fill = np.zeros((5, len(H))); tot = np.zeros(5)
for m in range(1, len(FORMS)):
    d1 = FORMS[m]; i_dec = int(DPOS[d1])
    r = month_paths(WVEC[d1] - WVEC[FORMS[m - 1]], BUCKET[d1], i_dec, BASE_LAG,␣
 ↪BASE_CAP)
    if r is None:
        continue
    act, start, finish, bk, need = r
    f_ = BASE_CAP / (1 - BASE_CAP)
    tgt = need * f_
    tt = np.minimum(i_dec + H + 1, len(CAL))
    got = np.minimum(tgt[None, :], (CUMVO[tt][:, act] - CUMVO[start, act][None,␣
 ↪:]) * f_)
    got = np.where((i_dec + H)[:, None] >= start[None, :], np.maximum(got, 0.
 ↪0), 0.0)
    for q in range(5):
        sel = bk == q
        if sel.any():
            fill[q] += got[:, sel].sum(axis=1)
```

```
            tot[q] += tgt[sel].sum()
FILL = pd.DataFrame((fill / np.where(tot > 0, tot, np.nan)[:, None]).T,␣
 ↪index=H, columns=BUCKET_NAMES)

fig, ax = plt.subplots(1, 2, figsize=(13, 4.3))
for q in range(5):
    ax[0].plot(H, FILL[BUCKET_NAMES[q]] * 100, color=OI[q],␣
 ↪label=BUCKET_NAMES[q])
ax[0].axvline(BASE_LAG, lw=0.8, ls="--", color="0.4"); ax[0].
 ↪set_xlabel("business days after the month-end decision")
ax[0].set_ylabel("% of target trade executed"); ax[0].set_title(f"Fill␣
 ↪trajectory (lag {BASE_LAG}bd, cap {BASE_CAP:.0%}, liquid-first)")
ax[0].legend(fontsize=8); ax[0].grid(alpha=0.25)

ORD = pd.concat([run_combo(BASE_LAG, BASE_CAP, o) for o in ["liquid_first",␣
 ↪"parallel", "illiquid_first"]])
piv_ord = ORD.reset_index().pivot(index="bucket", columns="order",␣
 ↪values="mean_days_from_decision").reindex(BUCKET_NAMES)
x = np.arange(5)
for i, o in enumerate(piv_ord.columns):
    ax[1].bar(x + (i - 1) * 0.27, piv_ord[o], width=0.27, color=OI[i], label=o)
ax[1].set_xticks(x); ax[1].set_xticklabels(BUCKET_NAMES, fontsize=8)
ax[1].set_ylabel("mean bd from decision to completion"); ax[1].
 ↪set_title("Ordering comparison")
ax[1].legend(fontsize=8); ax[1].grid(alpha=0.25, axis="y")
fig.tight_layout(); fig.savefig(OUT_DIR / "day5_base_case.png", dpi=150); plt.
 ↪show(); plt.close(fig)

print("\nmean business days from decision to completion, by ordering:")
print(piv_ord.round(1).to_string())
_lf, _pl = piv_ord.loc["Q1 illiquid", "liquid_first"], piv_ord.loc["Q1␣
 ↪illiquid", "parallel"]
print(f"\nliquid-first costs the illiquid quintile {_lf - _pl:+.1f} bd of extra␣
 ↪delay versus trading everything at once, "
      f"and buys the liquid quintile {piv_ord.loc['Q5 liquid', 'parallel'] -␣
 ↪piv_ord.loc['Q5 liquid', 'liquid_first']:+.1f} bd. "
      "Whether that trade is worth making is a cost question, not a timing one,␣
 ↪so it is left to the next pass.")
del D_base
gc.collect()
print(f"peak RSS so far: {peak_mb():,.0f} MB")
```

```
base case: AUM 100,000,000,000, lag 2bd, cap 10%, liquid-first
                n  mean_start_offset_bd  mean_days_exec  median_days_exec
p90_days_exec  mean_days_from_decision  median_days_from_decision
share_unresolved  share_truncated_by_sample_end  median_trade_yen
```

```
bucket
Q1 illiquid  5530                 22.23            3.03                1.0
6.0                     25.23                    18.0                0.01
0.0       1.874888e+07
Q2           4279                14.26            1.71                1.0
3.0                     15.97                    13.0                0.00
0.0       2.267514e+07
Q3           3957                 9.47            1.45                1.0
2.0                     10.93                     9.0                0.00
0.0       3.001728e+07
Q4           4049                 5.13            1.41                1.0
2.0                      6.54                     6.0                0.00
0.0       5.455272e+07
Q5 liquid    3607                 2.00            1.24                1.0
2.0                      3.24                     3.0                0.00
0.0       1.331477e+08
```




```
mean business days from decision to completion, by ordering:
order        illiquid_first  liquid_first  parallel
bucket
Q1 illiquid             5.1          25.2       5.1
Q2                     14.3          16.0       3.7
Q3                     19.5          10.9       3.4
Q4                     24.1           6.5       3.4
Q5 liquid              28.1           3.2       3.2
```

liquid-first costs the illiquid quintile +20.2 bd of extra delay versus trading everything at once, and buys the liquid quintile +0.0 bd. Whether that trade is worth making is a cost question, not a timing one, so it is left to the next pass.

peak RSS so far: 2,456 MB

1.6 6. Full grid: lag x participation cap x liquidity

`LAG_GRID` x `CAP_GRID` under liquid-first ordering, 63 combinations, five quintiles each. Two timing measures are reported and they answer different questions:

- **`days_exec`** = business days from a name's own first trading day to completion. This is pure capacity: it should depend strongly on the cap and on liquidity, and be **nearly flat in lag** (the only channel is that a different lag samples a different volume window). If it moves materially with lag, something is coupling the lag to capacity that should not be.
- **`days_from_decision`** = business days from the month-end decision to completion. This is the number that matters for implementation shortfall: lag, queue wait behind more liquid quintiles, and execution duration all land in it, and it should rise roughly one for one with lag.

Each `run_combo` call is discarded (`del` + `gc.collect()`) as soon as its small summary row is in hand, so the 63-combination loop never accumulates the per-name-month detail behind it.

[7]:
```
rows = []
for L in LAG_GRID:
    for c in CAP_GRID:
        rows.append(run_combo(L, c, "liquid_first"))
        gc.collect()
GRID = pd.concat(rows).reset_index()
del rows
gc.collect()
GRID.to_csv(OUT_DIR / "day5_lag_cap_liquidity_grid.csv", index=False)
print(f"grid: {len(GRID)} rows -> {OUT_DIR / 'day5_lag_cap_liquidity_grid.
 ↪csv'}")
print(f"peak RSS after the {len(LAG_GRID)}x{len(CAP_GRID)} sweep: {peak_mb():,.
 ↪0f} MB")

for b in BUCKET_NAMES:
    print(f"\nmean days_exec (own start -> done), {b}   rows = cap, cols =␣
 ↪lag(bd)")
    print(GRID[GRID["bucket"] == b].pivot(index="cap", columns="lag_bd",␣
 ↪values="mean_days_exec").round(1).to_string())

fig, axes = plt.subplots(1, 5, figsize=(22, 3.9))
vmax = GRID["mean_days_exec"].max()
for q, b in enumerate(BUCKET_NAMES):
    P = GRID[GRID["bucket"] == b].pivot(index="cap", columns="lag_bd",␣
 ↪values="mean_days_exec")
    im = axes[q].imshow(P.to_numpy(), aspect="auto", cmap="viridis_r", vmin=0,␣
 ↪vmax=vmax)
    axes[q].set_xticks(range(len(P.columns))); axes[q].set_xticklabels(P.
 ↪columns, fontsize=8)
    axes[q].set_yticks(range(len(P.index))); axes[q].set_yticklabels([f"{v:.
 ↪0%}" for v in P.index], fontsize=8)
```

```
    axes[q].set_xlabel("lag (bd)"); axes[q].set_title(b, fontsize=10)
    for i in range(P.shape[0]):
        for j in range(P.shape[1]):
            axes[q].text(j, i, f"{P.to_numpy()[i, j]:.0f}", ha="center",␣
 ↪va="center", fontsize=6.5,
                         color="w" if P.to_numpy()[i, j] > vmax * 0.45 else "k")
axes[0].set_ylabel("participation cap")
fig.suptitle("Mean business days to complete, measured from the name's own␣
 ↪start", y=1.03)
fig.tight_layout(); fig.savefig(OUT_DIR / "day5_days_exec_heatmaps.png",␣
 ↪dpi=150, bbox_inches="tight"); plt.show(); plt.close(fig)

fig, ax = plt.subplots(1, 3, figsize=(18, 4.3))
for q, b in enumerate(BUCKET_NAMES):
    g = GRID[(GRID["bucket"] == b) & (GRID["lag_bd"] == BASE_LAG)].
 ↪sort_values("cap")
    ax[0].plot(g["cap"] * 100, g["mean_days_exec"], marker="o", color=OI[q],␣
 ↪label=b)
    g2 = GRID[(GRID["bucket"] == b) & (GRID["cap"] == BASE_CAP)].
 ↪sort_values("lag_bd")
    ax[1].plot(g2["lag_bd"], g2["mean_days_from_decision"], marker="o",␣
 ↪color=OI[q], label=b)
    ax[2].plot(g2["lag_bd"], g2["mean_days_exec"], marker="o", color=OI[q],␣
 ↪label=b)
ax[0].set_xlabel("participation cap (%)"); ax[0].set_ylabel("mean days_exec");␣
 ↪ax[0].set_yscale("log")
ax[0].set_title(f"Capacity: exec days vs cap (lag {BASE_LAG}bd)")
ax[1].set_xlabel("lag (bd)"); ax[1].set_ylabel("mean days from decision");␣
 ↪ax[1].set_title(f"Total delay vs lag (cap {BASE_CAP:.0%})")
ax[2].set_xlabel("lag (bd)"); ax[2].set_ylabel("mean days_exec"); ax[2].
 ↪set_title("Audit: exec duration should be flat in lag")
for a in ax:
    a.legend(fontsize=7); a.grid(alpha=0.25)
fig.tight_layout(); fig.savefig(OUT_DIR / "day5_lag_cap_lines.png", dpi=150);␣
 ↪plt.show(); plt.close(fig)

UNRES = GRID.pivot_table(index=["bucket", "cap"], columns="lag_bd",␣
 ↪values="share_unresolved").loc[BUCKET_NAMES]
print(f"\nshare of name-months not completed within {MAX_EXEC_DAYS}bd of their␣
 ↪own start:")
print((UNRES * 100).round(2).to_string())
fig, ax = plt.subplots(figsize=(7.5, 4))
for q, b in enumerate(BUCKET_NAMES):
    g = GRID[(GRID["bucket"] == b) & (GRID["lag_bd"] == BASE_LAG)].
 ↪sort_values("cap")
```

```
    ax.plot(g["cap"] * 100, g["share_unresolved"] * 100, marker="o",␣
  ↪color=OI[q], label=b)
ax.set_xlabel("participation cap (%)"); ax.set_ylabel(f"% unresolved within␣
  ↪{MAX_EXEC_DAYS}bd")
ax.set_title(f"Names that never finish (AUM {AUM/1e8:,.0f} oku, lag␣
  ↪{BASE_LAG}bd)")
ax.legend(fontsize=8); ax.grid(alpha=0.25)
fig.tight_layout(); fig.savefig(OUT_DIR / "day5_unresolved.png", dpi=150); plt.
  ↪show(); plt.close(fig)

print(f"peak RSS so far: {peak_mb():,.0f} MB")
```

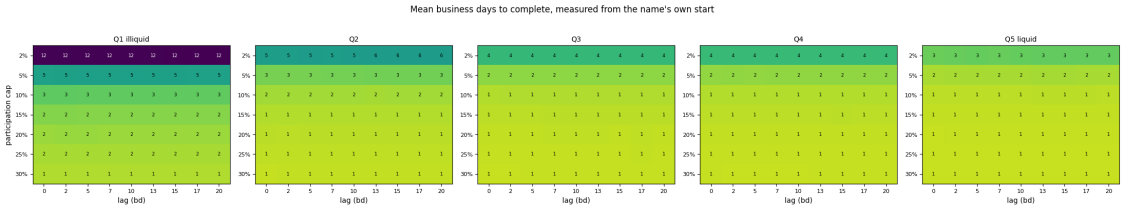
```
grid: 315 rows -> /home/ubuntu/work/output/day5_lag_cap_liquidity_grid.csv
peak RSS after the 9x7 sweep: 2,456 MB

mean days_exec (own start -> done), Q1 illiquid   rows = cap, cols = lag(bd)
lag_bd    0     2     5     7     10    13    15    17    20
cap
0.02    12.2  12.2  12.2  12.2  12.2  12.2  12.3  12.3  12.3
0.05     5.3   5.3   5.3   5.3   5.3   5.3   5.3   5.3   5.3
0.10     3.0   3.0   3.0   3.0   3.0   3.0   3.0   3.0   3.0
0.15     2.3   2.3   2.3   2.3   2.2   2.2   2.2   2.2   2.3
0.20     1.9   1.9   1.9   1.9   1.9   1.9   1.8   1.8   1.8
0.25     1.6   1.6   1.6   1.6   1.6   1.6   1.6   1.6   1.6
0.30     1.5   1.4   1.5   1.5   1.5   1.5   1.5   1.5   1.5

mean days_exec (own start -> done), Q2   rows = cap, cols = lag(bd)
lag_bd   0    2    5    7    10   13   15   17   20
cap
0.02    5.5  5.5  5.5  5.5  5.5  5.5  5.5  5.5  5.5
0.05    2.5  2.5  2.5  2.6  2.6  2.6  2.6  2.6  2.6
0.10    1.7  1.7  1.7  1.7  1.7  1.7  1.7  1.7  1.7
0.15    1.4  1.4  1.4  1.4  1.4  1.4  1.4  1.4  1.4
0.20    1.3  1.2  1.3  1.3  1.3  1.3  1.3  1.3  1.3
0.25    1.2  1.2  1.2  1.2  1.2  1.2  1.2  1.2  1.2
0.30    1.1  1.1  1.1  1.1  1.1  1.1  1.1  1.1  1.1

mean days_exec (own start -> done), Q3   rows = cap, cols = lag(bd)
lag_bd   0    2    5    7    10   13   15   17   20
cap
0.02    4.1  4.1  4.1  4.1  4.1  4.1  4.1  4.1  4.1
0.05    2.1  2.1  2.1  2.1  2.1  2.1  2.1  2.1  2.1
0.10    1.4  1.5  1.5  1.5  1.5  1.5  1.5  1.5  1.5
0.15    1.2  1.2  1.2  1.3  1.3  1.3  1.3  1.3  1.3
0.20    1.1  1.1  1.1  1.2  1.2  1.2  1.2  1.2  1.1
0.25    1.1  1.1  1.1  1.1  1.1  1.1  1.1  1.1  1.1
0.30    1.1  1.1  1.1  1.1  1.1  1.1  1.1  1.1  1.1
```

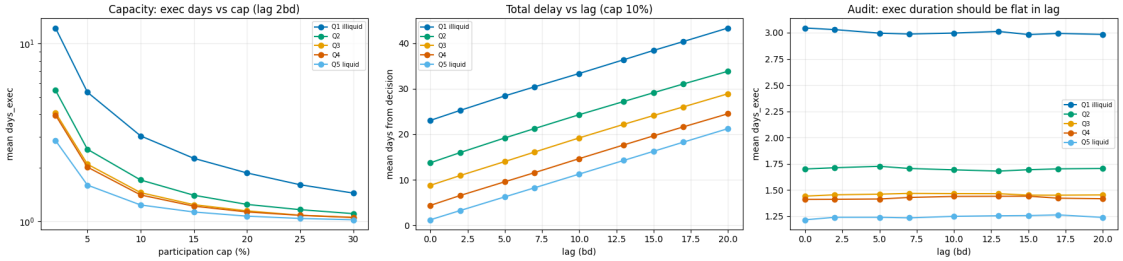
mean days_exec (own start -> done), Q4 rows = cap, cols = lag(bd)

lag_bd	0	2	5	7	10	13	15	17	20
cap									
0.02	3.9	4.0	4.0	4.0	4.0	4.0	4.0	4.0	4.0
0.05	2.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0
0.10	1.4	1.4	1.4	1.4	1.4	1.4	1.4	1.4	1.4
0.15	1.2	1.2	1.2	1.2	1.2	1.2	1.3	1.2	1.2
0.20	1.1	1.1	1.1	1.1	1.1	1.2	1.2	1.1	1.1
0.25	1.1	1.1	1.1	1.1	1.1	1.1	1.1	1.1	1.1
0.30	1.1	1.1	1.1	1.1	1.1	1.1	1.1	1.1	1.1

mean days_exec (own start -> done), Q5 liquid rows = cap, cols = lag(bd)

lag_bd	0	2	5	7	10	13	15	17	20
cap									
0.02	2.8	2.9	2.9	2.9	2.9	3.0	3.0	3.0	2.9
0.05	1.6	1.6	1.6	1.6	1.6	1.6	1.6	1.6	1.6
0.10	1.2	1.2	1.2	1.2	1.2	1.3	1.3	1.3	1.2
0.15	1.1	1.1	1.1	1.1	1.1	1.1	1.1	1.1	1.1
0.20	1.1	1.1	1.1	1.1	1.1	1.1	1.1	1.1	1.1
0.25	1.0	1.0	1.0	1.0	1.0	1.0	1.1	1.1	1.0
0.30	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0

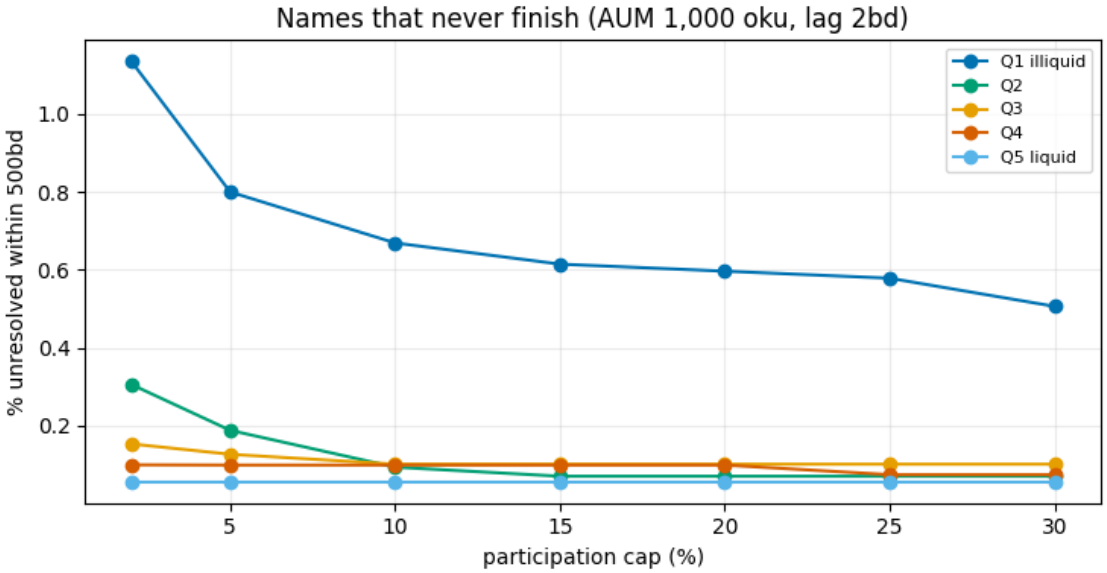




share of name-months not completed within 500bd of their own start:

lag_bd		0	2	5	7	10	13	15	17	20
bucket	cap									
Q1 illiquid	0.02	1.12	1.14	1.15	1.16	1.14	1.14	1.16	1.16	1.27
	0.05	0.80	0.80	0.80	0.80	0.82	0.84	0.84	0.88	0.97

	0.10	0.65	0.67	0.69	0.75	0.76	0.76	0.76	0.78	0.80
	0.15	0.61	0.61	0.63	0.65	0.67	0.71	0.73	0.75	0.76
	0.20	0.60	0.60	0.60	0.60	0.61	0.65	0.65	0.71	0.73
	0.25	0.52	0.58	0.58	0.58	0.58	0.63	0.65	0.65	0.67
	0.30	0.51	0.51	0.58	0.58	0.58	0.61	0.63	0.64	0.67
Q2	0.02	0.28	0.31	0.33	0.33	0.38	0.38	0.40	0.45	0.45
	0.05	0.16	0.19	0.26	0.28	0.31	0.31	0.33	0.33	0.35
	0.10	0.09	0.09	0.12	0.12	0.14	0.24	0.24	0.24	0.24
	0.15	0.07	0.07	0.07	0.09	0.09	0.16	0.19	0.19	0.19
	0.20	0.07	0.07	0.07	0.07	0.09	0.12	0.19	0.19	0.19
	0.25	0.07	0.07	0.07	0.07	0.07	0.09	0.16	0.19	0.19
	0.30	0.07	0.07	0.07	0.07	0.07	0.07	0.14	0.19	0.19
Q3	0.02	0.15	0.15	0.15	0.15	0.15	0.20	0.23	0.23	0.23
	0.05	0.13	0.13	0.13	0.13	0.13	0.13	0.13	0.15	0.15
	0.10	0.10	0.10	0.10	0.10	0.13	0.13	0.13	0.13	0.13
	0.15	0.10	0.10	0.10	0.10	0.10	0.13	0.13	0.13	0.13
	0.20	0.10	0.10	0.10	0.10	0.10	0.13	0.13	0.13	0.13
	0.25	0.10	0.10	0.10	0.10	0.10	0.13	0.13	0.13	0.13
	0.30	0.10	0.10	0.10	0.10	0.10	0.13	0.13	0.13	0.13
Q4	0.02	0.10	0.10	0.10	0.10	0.15	0.17	0.17	0.17	0.17
	0.05	0.10	0.10	0.10	0.10	0.10	0.10	0.15	0.15	0.15
	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.12	0.15
	0.15	0.07	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.15
	0.20	0.07	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.15
	0.25	0.07	0.07	0.07	0.10	0.10	0.10	0.10	0.10	0.15
	0.30	0.07	0.07	0.07	0.07	0.10	0.10	0.10	0.10	0.15
Q5 liquid	0.02	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.11
	0.05	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.11
	0.10	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.11
	0.15	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.11
	0.20	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.11
	0.25	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.11
	0.30	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.06	0.11




```
peak RSS so far: 2,456 MB
```

1.7 7. Reading it, and what this deliberately does not do

What the grid answers. For a 1000 fund running this monthly low-PBR list, how many trading days each liquidity quintile actually needs at a given participation cap, and how much calendar time sits between the month-end decision and a completed position once the lag and the liquid-first queue are added.

Checks to make before quoting any of it: - `days_exec` should be close to flat across the lag axis within a bucket. A visible slope means the lag is changing available capacity, which it should not. - `days_from_decision` minus `days_exec` minus `start_offset_bd` should be zero by construction; the start offset is where the lag and the queue wait show up separately. - Watch `share_unresolved` in Q1 at low caps. At 1000 a 1% weight is 10 in one name; against a Q1 median ADV near 2 that is many days even at a 30% cap, and at 2% it may not finish at all. If Q1 unresolved is large, the honest conclusion is that this AUM cannot hold the illiquid tail at that cap, which is a result, not a bug.

Known simplifications, all deliberate for this pass: - No execution cost, no spread, no impact. Timing only. - No intraday split and no clustering indicator. Both attach later, at the point where a day's slice gets priced. - Trades are the undrifted month-over-month weight change; a drift-consistent fund engine changes the trade sizes slightly but not the ordering or the capacity arithmetic. - Participation caps are applied to the same day's printed volume (mild foresight, same convention as Day 3). An ADV-based cap is a one-line change and would be the conservative variant. - Volume is close times shares from the returns file, so it is a proxy for , not the exchange's own figure.

If this OOMs again, the `peak RSS so far` printouts after §1/§2/§4/§5/§6 pinpoint which section did it. Rerun just that section with `tracemalloc`, or wrap the suspect line with `memory_profiler`'s `%mprun`, rather than re-guessing across the whole notebook.